

Predavanje 2 — Ljuska, procesi i shell skripte

Programiranje za UNIX

Damir Krstinić Maja Braović

FESB — Sveučilište u Splitu

- ▶ UNIX nastaje 1969. kao **jednostavniji odgovor na Multics**; oskudno sklopovlje, izostanak proračuna i besplatno dijeljenje s izvornim kodom oblikovali su sve ostalo.
- ▶ Danas ga nalazimo na poslužiteljima, telefonima, ugradbenim uređajima i **svim** superračunalima s liste TOP500.
- ▶ Arhitektura je slojevita; **sistemske pozivi su jedino sučelje** prema jezgri.
- ▶ **Sve je datoteka** — pozivi open/read/write/close za sve resurse.
- ▶ Jedno stablo s korijenom /, bez slova diskova i bez ekstenzija.
- ▶ Prava: tri skupine $\times$ tri prava; x daje pravo izvršavanja, a x na direktoriju mogućnost otvaranja datoteka u njemu.

1. **Naredbena ljuska** — osnovne naredbe i način rada.
2. **Preusmjeravanje i ulančavanje** — kako programi razmjenjuju podatke.
3. **Programi i procesi** — što se događa kad pokrenemo program.
4. **Shell skripte** — ljuska kao programski jezik.

1.

Naredbena ljuska

- ▶ Ljuska je **interpreter naredbenog retka**: čita naredbu sa standardnog ulaza, izvršava je i ispisuje rezultat.
- ▶ Nije dio jezgre — to je običan korisnički program.
- ▶ Format naredbe:

naredba [opcije] [argumenti]

- ▶ Opcije se pišu s crticom (-l), duge opcije s dvije (--all), a više kratkih opcija može se spojiti: `ls -la` je isto što i `ls -l -a`.
- ▶ **UNIX razlikuje velika i mala slova** — u imenima naredbi, opcija i datoteka: `ls` i `LS` nisu ista naredba, `-r` i `-R` nisu ista opcija.

Uputa: `man <naredba>`

Razlike među ljuskama

Ljuska je **običan program**, a ne dio jezgre — pa ih na sustavu može biti i više, i svaki korisnik može birati svoju.

Ljuska	Opis
sh	<i>Bourne shell</i> , izvorna UNIX ljuska; danas standard za skripte
bash	<i>Bourne Again Shell</i> , proširuje sh; uobičajena na Linuxu
dash	mala i brza sh-kompatibilna ljuska za systemske skripte
ksh	<i>Korn shell</i> , čest na komercijalnim UNIX sustavima
csh, tcsh	<i>C shell</i> , sintaksa nalik C-u; drukčija od sh obitelji
zsh	proširena bash-kompatibilna ljuska; zadana na macOS-u

Koju trenutno koristimo: `echo $SHELL`, popis dostupnih: `cat /etc/shells`.

Kada razlike postaju važne

- ▶ **U interaktivnom radu gotovo nikad** — naredbe (`ls`, `cp`, `grep`), putanje, prava i preusmjeravanje izlaza rade jednako u svim ljuskama.
- ▶ **Razlike se pojavljuju kod:**
 - ▶ sintakse skripti — varijable, uvjeti, petlje (`bash` i `csh` bitno se razlikuju),
 - ▶ preusmjeravanja pogreške (`stderr`),
 - ▶ konfiguracijskih datoteka i varijabli okruženja.
- ▶ **O tome treba voditi računa kada:**
 - ▶ pišemo skriptu — prvi redak (*shebang*) određuje interpreter i mora odgovarati sintaksi,
 - ▶ radimo na tuđem ili udaljenom sustavu gdje zadana ljuska nije ona na koju smo navikli,
 - ▶ preuzimamo primjer s interneta — radi u `bash`-u, ne mora u `csh`-u.

U primjerima skripti koje ćemo obraditi koristimo **bash**.

Ugrađene naredbe i programi

Ljusci možemo zadati dvije vrste naredbi:

- ▶ **Vanjski programi** — svakoj odgovara izvršna datoteka na disku (/bin/ls, /usr/bin/grep). Pokretanjem nastaje **novi proces**.
- ▶ **Ugrađene naredbe** (*shell built-ins*) — implementirane u samoj ljusci, bez izvršne datoteke. Ne stvaraju novi proces.

Klasičan primjer ugrađene naredbe je `cd`: ona mijenja radni direktorij **same ljuske**, pa bi kao zaseban proces bila besmislena.

Koja je koja provjerava se naredbom `type`:

```
$ type cd
cd is a shell builtin
$ type ls
ls is /usr/bin/ls
```


Kretanje po datotečnom sustavu

Naredba	Opis	Primjer
<code>pwd</code>	ispis trenutnog direktorija	<code>pwd</code>
<code>cd</code>	promjena direktorija	<code>cd /etc, cd .., cd</code>
<code>ls</code>	ispis sadržaja direktorija	<code>ls -la /home</code>

Korisne opcije naredbe `ls`:

- ▶ `-l` — dugi format (prava, vlasnik, veličina, vrijeme),
- ▶ `-a` — i skrivene datoteke (one čije ime počinje točkom),
- ▶ `-t` — sortirano po vremenu izmjene, `-r` obrnuto,
- ▶ `-h` — veličine u čitljivom obliku (4.0K, 12M).

Rad s datotekama i direktorijima

Naredba	Opis
cp	kopiranje (-r rekurzivno, -i s potvrdom, -f bezuvjetno)
mv	premještanje i preimenovanje
rm	brisanje (-R rekurzivno, -f bez upita)
mkdir	stvaranje direktorija (-p i svi roditelji)
rmdir	brisanje praznog direktorija
touch	stvaranje prazne datoteke ili osvježavanje vremena

U UNIX-u nema “koša za smeće” — brisanje je trajno.

Oprez s rm

```
$ rm -Rf stari_dir/
```

- ▶ -R briše rekurzivno cijelo stablo, -f ne postavlja nijedno pitanje.
 - ▶ Kombinacija je nužna u svakodnevnom radu, ali i najčešći uzrok nepovratnog gubitka podataka.
 - ▶ Posebno opasno: `rm -Rf /` ili slučajan razmak u `rm -Rf *.o` umjesto `rm -Rf *.o`.
-

Prije pritiska na Enter dvaput pročitajte što ste utipkali. Kod brisanja nema poništavanja.

Pregled sadržaja datoteka

Naredba	Opis
<code>cat</code>	ispis cijelog sadržaja na standardni izlaz
<code>less</code>	pregled po stranicama (q za izlaz, / za pretragu)
<code>head</code>	prvih 10 redaka (-n 25 za drugi broj)
<code>tail</code>	zadnjih 10 redaka (-f prati datoteku dok raste)
<code>wc</code>	broj redaka, riječi i znakova (-l, -w, -c)

`tail -f` je nezaobilazan pri praćenju dinamičkih log datoteka poslužitelja u stvarnom vremenu.

Pretraživanje

- ▶ **grep** — ispisuje retke koji odgovaraju uzorku:

```
$ grep "ERROR" program.log
$ grep -i -n "greska" *.txt      # bez razlike u veličini slova, s brojem retka
$ grep -r "TODO" .              # rekurzivno kroz stablo
$ grep -C 2 "ERROR" program.log  # i po 2 retka prije i poslije
```

- ▶ **find** — traži datoteke po imenu, tipu, veličini ili vremenu:

```
$ find . -name "*.c"
$ find /home -type d -name "projekti"
$ find . -mtime -7           # izmijenjeno u zadnjih 7 dana
```

Ostale korisne naredbe

Naredba	Opis
echo	ispis teksta ili vrijednosti varijable
date	datum i vrijeme
who, id	tko je prijavljen; identitet i grupe korisnika
du, df	zauzeće direktorija; slobodan prostor na disku
sort, uniq	sortiranje; uklanjanje susjednih duplikata
chmod, chown	promjena prava i vlasništva
clear, exit	čišćenje zaslona; izlaz iz ljuske

Priručnik: man

- ▶ `man <naredba>` otvara priručnik dostupan izravno u terminalu, bez internetske veze.
- ▶ Navigacija strelicama i tipkom Space, izlaz tipkom q, pretraga s /uzorak.
- ▶ Stranice su podijeljene u sekcije:
 - ▶ **1** — korisničke naredbe: `man 1 ls`,
 - ▶ **2** — sistemski pozivi: `man 2 open`,
 - ▶ **3** — funkcije biblioteka: `man 3 printf`.
- ▶ Broj sekcije navodimo kad isto ime postoji na više razina — `printf` je i naredba i funkcija.
- ▶ man stranice uvijek odgovaraju verziji alata na tom računalu; internetski izvori to ne jamče.

Pokretanje programa u pozadini

Ljuska po zadanom čeka da program završi. Znakom & pokrećemo ga **u pozadini**:

```
$ tar -czf backup.tar.gz /home/dkrst &  
[2] 3963  
$
```

- ▶ [2] je redni broj posla u ljusci, 3963 je PID procesa.
- ▶ jobs — popis poslova, fg %2 — vraćanje u prvi plan, bg %2 — nastavak u pozadini.
- ▶ Ctrl+Z suspendira program u prvom planu; bg ga zatim nastavlja u pozadini.

- ▶ Na UNIX-u grafičko sučelje **nije dio operacijskog sustava** — to je skup običnih korisničkih programa iznad iste jezgre i istih sistemskih poziva.
- ▶ Slaže se od nekoliko nezavisnih slojeva koje korisnik bira:
 - ▶ **grafički poslužitelj** — X Window System (X11), danas sve češće Wayland,
 - ▶ **upravitelj prozora** (*window manager*) — crta okvire, pomiče i slaže prozore,
 - ▶ **radna okolina** (*desktop environment*) — GNOME, KDE, XFCE: upravitelj prozora, ploče, upravitelj datoteka i alati u jednoj cjelini.
- ▶ Svaki se sloj može zamijeniti ili potpuno izostaviti. Poslužitelji redovito rade **bez ijednog od njih**, a njima se upravlja isključivo ljuskom.

X: klijent i poslužitelj

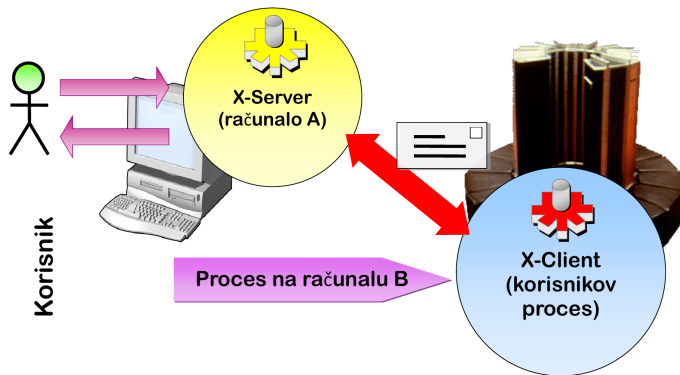
- ▶ **X poslužitelj** radi na računalu koje ima zaslon, tipkovnicu i miša; on jedini upravlja grafičkim sklopovljem.
- ▶ **X klijent** je program s grafičkim sučeljem. Klijent poslužitelju šalje zahtjeve “nacrtaj prozor”, a prima događaje tipkovnice i miša.
- ▶ Ključno: klijent i poslužitelj **ne moraju biti na istom računalu**. Program se izvršava na udaljenom stroju, a prikazuje se lokalno:

```
$ ssh -X korisnik@posluzitelj
```

```
$ xclock &
```

- ▶ Uočite obrnutu perspektivu: *poslužitelj* je računalu ispred kojeg sjedite.

X Windows sustav



X poslužitelj i X klijent na različitim računalima

Razlika prema Windowsima

	UNIX	Windows
Položaj GUI-ja	korisnički programi iznad jezgre	dio sustava, dijelom u jezgri
Može li se izostaviti	da, sustav radi bez njega	ne u punom smislu
Izbor sučelja	GNOME, KDE, XFCE i drugi	jedno, zadano
Rad na daljinu	ugrađen: prikaz na drugom računalu	naknadno dodan (RDP)
Uobičajen rad na poslužitelju	bez GUI-ja, preko ljuske	često s grafičkim sučeljem

UNIX sustavom moguće je u potpunosti upravljati **bez grafičkog sučelja**, putem ljuske i sistemskih poziva.

2.

Preusmjeravanje i ulančavanje

Tri standardna toka

Svaki program pri pokretanju dobiva tri otvorena kanala:

- ▶ **stdin** — standardni ulaz, prema zadanom tipkovnica,
- ▶ **stdout** — standardni izlaz, prema zadanom zaslon,
- ▶ **stderr** — standardni izlaz za greške, također zaslon, ali **odvojen tok**.

Ljuska te tokove može prije pokretanja programa spojiti na nešto drugo — datoteku ili drugi program. **Sam program se pritom ne mijenja** i ne zna za promjenu.

Razdvojenost stdout i stderr upravo zato ima smisla: rezultat možemo spremiti u datoteku, a greške i dalje vidjeti na zaslonu.

Operatori preusmjerenja

Operatori se donekle razlikuju među ljuskama; ovo su operatori **bash ljuske**:

Operator	Opis
>	preusmjeri stdout u datoteku (briše postojeći sadržaj)
>>	dodaj stdout na kraj datoteke
<	čitaj stdin iz datoteke
2>	preusmjeri stderr u datoteku
2>>	dodaj stderr na kraj datoteke
&>	preusmjeri stdout i stderr zajedno
2>&1	spoji stderr na stdout

U csh/tcsh ljusci stdout i stderr ne mogu se razdvojiti tako jednostavno: oba se zajedno preusmjeravaju operatorom >&, a za razdvajanje je potrebna zaobilazna konstrukcija. Ljuske sh, bash, ksh i zsh koriste operatore iz tablice.

Preusmjerenje izlaza

```
$ ls -la > popis.txt
$ cat popis.txt
total 8
drwxr-xr-x 4 dkrst users 168 2026-03-11 18:04 .
drwxr-xr-x 11 dkrst users 352 2026-03-11 12:38 ..
-rw-r--r-- 1 dkrst users 33 2026-03-11 12:58 dat1.txt
-rw-r--r-- 1 dkrst users 33 2026-03-11 16:12 dat2.txt
-rw-r--r-- 1 dkrst users 0 2026-03-11 18:04 popis.txt
```

Uočite: `popis.txt` već postoji i prazan je — ljuska datoteku stvara **prije** pokretanja programa.

Odvajanje grešaka

```
$ ls /home /nepostoji > rezultati.txt 2> greske.txt
```

```
$ cat rezultati.txt
```

```
/home:
```

```
marko ana petar
```

```
$ cat greske.txt
```

```
ls: cannot access '/nepostoji': No such file or directory
```

Datoteka **/dev/null** je “crna rupa” jezgre — sve što se u nju upiše nestaje:

```
$ naredba 2> /dev/null          # zanemari greske
```

```
$ naredba > /dev/null 2>&1      # zanemari sve, samo izvrši
```

Ulančavanje naredbi

Operator | spaja stdout jedne naredbe izravno na stdin druge:

```
$ cat program.log | grep "ERROR" | wc -l  
42
```

- ▶ cat šalje sadržaj datoteke dalje,
- ▶ grep propušta samo retke s riječi ERROR,
- ▶ wc -l broji retke.

Nijedan od tri programa ne zna ništa o ostalima — a zajedno rješavaju konkretan zadatak.

Lanci u praksi

Zapis u datoteci `auth.log` izgleda ovako — korisničko ime je **peto polje**:

```
Mar 11 18:04 ERROR marko failed password from 10.0.0.7
```

Različita korisnička imena koja su izazvala grešku:

```
$ grep "ERROR" auth.log | awk '{print $5}' | sort | uniq
admin
ana
marko
```

- ▶ `awk '{print $5}'` izdvaja peto polje retka (polja dijeli razmak),
- ▶ `sort` poreda imena,
- ▶ `uniq` uklanja duplikate — traži **susjedne**, pa se prije njega uvijek sortira.

Ovo je UNIX filozofija na djelu: mali programi koji rade jednu stvar dobro kombiniraju se u moćne lance obrade.

Imenovani cjevovod

- ▶ Cjevovod stvoren operatorom | je **anoniman** — postoji samo dok naredbe traju.
- ▶ **Imenovani cjevovod** (FIFO) trajan je objekt u datotečnom sustavu:

```
$ mkfifo cijev
```

```
$ ls -l cijev
```

```
prw-r--r-- 1 dkrtst users 0 2026-03-11 18:20 cijev
```

- ▶ Oznaka tipa je p. Otvoriti ga može bilo koji proces koji zna putanju, i nakon što je proces koji ga je stvorio završio.
- ▶ Detaljno u poglavlju o komunikaciji između procesa.

Demo: preusmjeravanje i lanci

Cilj: pokazati da isti program radi jednako, neovisno o tome odakle čita i kamo piše.

```
wc -l < /etc/passwd  
ls /etc | wc -l  
ls /etc /nepostoji > out.txt 2> err.txt ; cat err.txt  
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

Poanta: ljuska spaja tokove, programi ostaju isti.

3.

Programi i procesi

Program i proces

- ▶ **Program** je izvršna datoteka na disku:
 - ▶ izvorni kôd preveden i povezan u strojne naredbe, ili
 - ▶ skup naredbi koji se interpretira pri pokretanju (npr. shell skripta).
- ▶ **Proces** je aktivni entitet u memoriji koji se izvršava na sklopovlju.
- ▶ Program je statičan; **postaje proces tek kad ga pokrenemo.**
- ▶ Isti program može istovremeno biti pokrenut kao više nezavisnih procesa.

Identifikacija procesa

Pri stvaranju procesa jezgra mu dodjeljuje resurse, okruženje, ovlasti i oznake:

- ▶ **PID** (*Process ID*) — jedinstveni broj procesa.
- ▶ **PPID** (*Parent Process ID*) — PID procesa koji ga je pokrenuo.
- ▶ **UID, GID** — korisnik i grupa vlasnika procesa; određuju njegove ovlasti.
- ▶ **EUID, EGID** — *efektivni* korisnik i grupa; koriste se za privremeno podizanje ovlasti.

Klasičan primjer efektivnih ovlasti je `passwd`: obični korisnik njime mijenja vlastitu lozinku iako nema pravo pisanja u `/etc/shadow`.

Pregled procesa: ps

```
$ ps -ef | head -5
UID          PID  PPID  C  STIME TTY          TIME CMD
root           1      0  0  Mar01 ?        00:00:42 /sbin/init
root           2      0  0  Mar01 ?        00:00:00 [kthreadd]
dkrst       14567 14566  0  09:15 pts/0    00:00:00 -bash
dkrst       25016 14567  0  12:30 pts/0    00:00:00 ./program
```

- ▶ `ps -ef` — svi procesi u sustavu, `ps aux` — s postotkom procesora i memorije.
- ▶ Uočite stupac PPID: `./program` je pokrenut iz ljuske (14567), a ljuska iz procesa 14566.
- ▶ `ps tree` prikazuje isto kao stablo, `top` i `htop` u stvarnom vremenu.

Stablo procesa

- ▶ Novi proces uvijek nastaje iz **postojećeg** procesa, sistemskim pozivom `fork()`.
- ▶ Zato su svi procesi u sustavu organizirani kao **stablo**, s procesom `init` (PID 1) u korijenu.
- ▶ Kad roditelj završi prije djeteta, dijete nasljeđuje `init` kao novog roditelja.

```
init(1) — sshd(892) — bash(14567) — program(25016)
      |
      |— cron(901)
      |— syslogd(915)
```

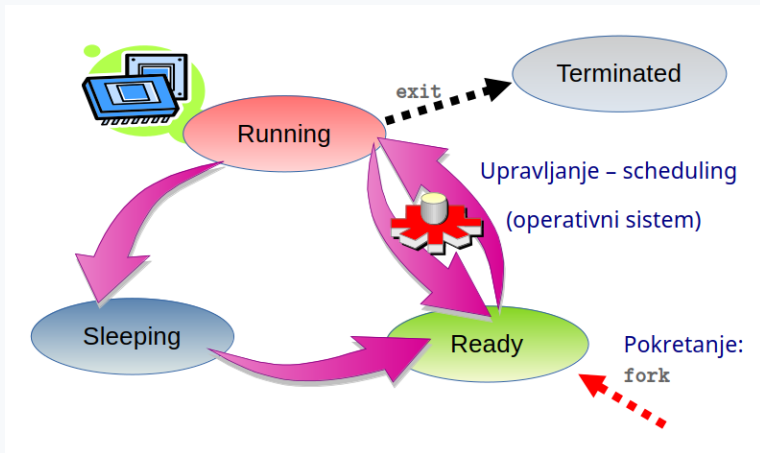
Životni ciklus procesa

Proces tijekom života prolazi kroz nekoliko stanja:

- ▶ **Ready** — spreman za izvršavanje, čeka procesor.
- ▶ **Running** — trenutno se izvršava.
- ▶ **Blocked** (*sleeping*) — čeka na događaj (podatke s diska, mreže, tipkovnice).
- ▶ **Terminated** — završio, čeka da roditelj pokupi izlazni status.

O prelascima između *ready* i *running* odlučuje **raspoređivač** (*scheduler*) jezgre. Proces sam ne bira kad će dobiti procesor.

Životni ciklus procesa



Životni ciklus procesa (izvor: skripta, slika 5.1)

Stvaranje i završetak

- ▶ **fork()** — stvara novi proces kao kopiju postojećeg.
- ▶ **exec()** — zamjenjuje memorijsku sliku procesa novim programom.
- ▶ **exit()** — završava proces uz izlazni status.
- ▶ **wait()** — roditelj preuzima izlazni status djeteta.

Pokretanje programa iz ljuske zapravo je slijed `fork()` pa `exec()`: ljuska se udvostruči, a kopija se pretvori u traženi program.

Detaljna obrada slijedi u poglavljima o okruženju procesa i signalima.

Vrste procesa

Prema odnosu:

- ▶ **roditelj** (*parent*) — proces koji pokreće druge,
- ▶ **dijete** (*child*) — pokrenut iz roditelja, u PPID nosi njegov PID.

Prema stanju i ulozi:

- ▶ **running** — aktivno se izvršava,
- ▶ **sleeping / idle** — neaktivan, čeka događaj,
- ▶ **stopped** — zaustavljen (npr. Ctrl+Z),
- ▶ **orphan** — roditelj je završio, proces radi dalje i nasljeđuje init,
- ▶ **daemon** — namjerno pozadinski proces, nije vezan ni uz jedan terminal,
- ▶ **zombie** — završio, ali izlazni status još nije pokupljen.

Daemoni i zombiji

- ▶ **Daemon** obavlja sistemske ili mrežne usluge neovisno o tome je li itko prijavljen. Imena im tradicionalno završavaju slovom d: sshd (mrežne veze), syslogd (logovi), crond (raspoređeni poslovi).
- ▶ **Zombie** nastaje kad proces završi, a roditelj ne pokupi njegov izlazni status. Proces više ne troši memoriju ni procesor, ali zauzima zapis u tablici procesa.
 - ▶ Jezgra taj zapis čuva jer netko još može zatražiti izlazni status.
 - ▶ Mnogo zombija u sustavu znak je greške u programu roditelja.
- ▶ Oba pojma vraćaju se detaljno u poglavlju o okruženju procesa.

Prioritet procesa

- ▶ Prioritet određuje koliko često i koliko dugo proces dobiva procesor.
- ▶ Određuje se na temelju vrijednosti **nice value**, u rasponu od **-20 do 19**; zadana vrijednost je 0.
- ▶ Veći broj znači **niži** prioritet — proces je “ljubazniji” prema ostalima jer traži manje procesorskog vremena.
- ▶ Obični korisnik smije prioritet samo **snizavati**; povisiti ga (negativne vrijednosti) može jedino root.

```
$ nice -n 10 ./dugotrajni_posao      # pokreni sa snizenim prioritetom  
$ renice -n 5 -p 1111                # promijeni prioritet pokrenutom procesu
```

Trenutne vrijednosti vide se u stupcu NI naredbe top.

- ▶ **Signal** je obavijest o asinkronom događaju koju jezgra šalje procesu — najjednostavniji oblik komunikacije s procesom i među procesima.
- ▶ Proces može signal **uhvatiti** (izvršiti vlastitu funkciju), **ignorirati** ili prepustiti predefiniranoj akciji, koja je najčešće prekid izvršavanja.
- ▶ Signal može poslati jezgra (greška u programu), korisnik (tipkovnicom) ili drugi proces (naredbom `kill`).
- ▶ Svaki signal ima broj, ali u programima uvijek koristimo simbolička imena iz `<signal.h>`.

Najčešći signali

Signal	Broj	Značenje
SIGHUP	1	prekinuta je sesija ili zatvoren terminal
SIGINT	2	korisnik je pritisnuo Ctrl+C
SIGKILL	9	bezuvjetni prekid — ne može se uhvatiti
SIGSEGV	11	pristup nedozvoljenoj memoriji
SIGTERM	15	pristojan zahtjev za prekid
SIGSTOP	19	zaustavljanje — ne može se uhvatiti ; nastavak s SIGCONT
SIGTSTP	20	korisnik je pritisnuo Ctrl+Z

Puni popis: `man 7 signal`.

4. **Shell skripte**

Ljuska kao programski jezik

- ▶ Niz naredbi zapisan u tekstualnoj datoteci s pravom izvršavanja postaje **shell skripta** — cjelovit program.
- ▶ Koristimo je za automatizaciju: sigurnosne kopije, obradu podataka, pokretanje i nadzor programa, administraciju sustava.
- ▶ Prvi redak je **shebang** — direktiva kojom jezgra bira interpreter:

`#!/bin/bash`

- ▶ Komentari počinju znakom `#` i traju do kraja retka.
- ▶ Skripta se pokreće kao i svaki drugi program:

```
$ chmod +x skripta.sh
```

```
$ ./skripta.sh
```

Prva skripta

```
#!/bin/bash
# Najjednostavnija shell skripta - ispisuje pozdrav korisniku.
```

```
echo "Pozdrav, $USER!"
echo "Trenutni radni direktorij: $(pwd)"
echo "Datum i vrijeme: $(date)"
```

- ▶ **\$USER** — varijabla okruženja koju ljuska sama postavlja.
- ▶ **\$(naredba)** — naredbena supstitucija: ljuska izvrši naredbu i na to mjesto umetne njezin izlaz.

Primjer `pozdrav.sh` iz repozitorija skripte.

Variable

```
ime="Kuzma"           # bez razmaka oko znaka =  
broj=42  
echo "$ime ima $broj godine"
```

- ▶ Vrijednost se čita znakom \$ ispred imena.
- ▶ Varijable se **uvijek navode u dvostrukim navodnicima**: bez njih vrijednost s razmakom raspada se na više argumenata.
- ▶ \${ime}_dodatak — vitičaste zagrade kad se ime nadovezuje na tekst.
- ▶ Ljuska ne poznaje tipove: sve je znakovni niz.

Argumenti naredbenog retka

Oznaka	Značenje	Analogija u C-u
\$0	ime skripte	argv[0]
\$1, \$2, ...	prvi, drugi argument	argv[1], argv[2]
\$#	broj argumenata	argc - 1
\$@	svi argumenti	—
\$?	izlazni status zadnje naredbe	—

```
if [ $# -ne 1 ]; then
    echo "Korištenje: $0 <direktorij>"
    exit 1
fi
```

Uvjetno grananje

```
if [ -f "$1" ]; then
    echo "$1 je datoteka"
elif [ -d "$1" ]; then
    echo "$1 je direktorij"
else
    echo "$1 ne postoji"
fi
```

- ▶ Razmaci unutar uglatih zagrada **obavezni** su — [je zapravo naredba.
- ▶ Testovi datoteka: -f obična datoteka, -d direktorij, -e postoji, -r/-w/-x prava, -z prazan niz. Znak ! negira test.
- ▶ Usporedba brojeva: -eq, -ne, -lt, -gt, -le, -ge. Usporedba nizova: = i !=.

Petlje

```
for i in $(seq 1 5); do
    echo "  $i"
done
```

```
for f in *.txt; do
    echo "Obrađujem $f"
done
```

```
while [ $a -gt 0 ]; do
    echo $a
    (( a-- ))
done
```

Lista u for petlji može biti zadana izravno, generirana naredbom ili dobivena razrješavanjem uzorka imena datoteka.

Primjer: ispis argumenata

```
#!/bin/bash
# Ispisuje sve argumente naredbenog retka, jedan po retku.
```

```
echo "Broj argumenata: $#"  
i=1  
for arg in "$@"; do  
    echo "  argument $i: $arg"  
    (( i++ ))  
done
```

```
$ ./argumenti.sh prvi drugi "treći s razmakom"  
Broj argumenata: 3  
  argument 1: prvi  
  argument 2: drugi  
  argument 3: treći s razmakom
```

Navodnici oko "\$@" bitni su: bez njih bi se treći argument raspao na tri.

Primjer: brojac.sh

```
#!/bin/bash
# Broji od 1 do N (N se daje kao argument, ili 5 ako nije zadan).

if [ $# -eq 0 ]; then
    n=5
else
    n=$1
fi

echo "Brojim od 1 do $n..."
for i in $(seq 1 $n); do
    echo "  $i"
done
echo "Gotovo!"
```

Izlazni status

- ▶ Svaka naredba pri završetku vraća **izlazni status**: 0 znači uspjeh, sve ostalo (1–255) neku vrstu greške.
- ▶ Status zadnje izvršene naredbe čita se iz \$?:

```
tar -czf "$arhiva" "$dir"
if [ $? -eq 0 ]; then
    echo "Arhiva uspješno stvorena."
else
    echo "Greška pri stvaranju arhive."
    exit 3
fi
```

- ▶ Skripta vlastiti status postavlja naredbom `exit N`. Time postaje upotrebljiva unutar drugih skripti i lanaca.

Primjer: backup.sh

```
#!/bin/bash
# Stvara komprimiranu arhivu zadanog direktorija s timestampom.

if [ $# -ne 1 ]; then
    echo "Korištenje: $0 <direktorij>"
    exit 1
fi
dir=$1
if [ ! -d "$dir" ]; then
    echo "Greška: '$dir' nije direktorij."
    exit 2
fi
timestamp=$(date +"%Y-%m-%d_%H-%M")
arhiva="$(basename "$dir")_${timestamp}.tar.gz"
tar -czf "$arhiva" "$dir"
```

Puni kôd s provjerom rezultata je u repozitoriju skripte.

bash i csh

Skripta je vezana uz ljusku za koju je pisana. Najčešće su bash i csh:

Operacija	bash	csh
Shebang	<code>#!/bin/bash</code>	<code>#!/usr/bin/csh</code>
Dodjela varijable	<code>ime="Kuzma"</code>	<code>set ime="Kuzma"</code>
Uvjet	<code>if [\$a = \$b]; then</code>	<code>if (\$a == \$b) then</code>
Kraj if bloka	<code>fi</code>	<code>endif</code>
for petlja	<code>for i in lista; do</code>	<code>foreach i (lista)</code>
Kraj petlje	<code>done</code>	<code>end</code>
Izlazni status	<code>\$?</code>	<code>\$status</code>
Argumenti	<code>\$1, \$2, ...</code>	<code>\$argv[1], \$argv[2], ...</code>

U nastavku kolegija koristimo bash.

Što smo naučili

- ▶ Ljuska pokreće vanjske programe i vlastite ugrađene naredbe; `man`: pomoć za bilo koju UNIX naredbu (programe i ugrađene naredbe).
- ▶ Tri standardna toka mogu se preusmjeriti u datoteke ili ulančati operatorom `|` — programi se pritom ne mijenjaju.
- ▶ Program je datoteka, proces je program u izvođenju; svaki proces ima PID, roditelja i vlasnika.
- ▶ Procesi nastaju pozivom `fork()`, mijenjaju kôd pozivom `exec()` i završavaju pozivom `exit()`.
- ▶ Signalima upravljamo procesima: `SIGTERM` pristojno, `SIGKILL` bezuvjetno.
- ▶ Shell skripta je program: varijable, argumenti, grananje, petlje i izlazni status.

Prevođenje i povezivanje programa

- ▶ od izvornog koda do izvršne datoteke,
- ▶ prevodilac gcc i njegove osnovne opcije,
- ▶ prevođenje u jednom i u dva koraka,
- ▶ program raspoređen u više datoteka.

Do tada: napišite barem jednu skriptu koja rješava neki vaš stvarni, svakodnevni zadatak.